Evaluating k-Nearest Neighbors for Categorical Prediction: Technical Implementation, Real-World Application, and Ethical Implications

Tuguinay, Nanelle
College of Computer Studies and Engineering
Jose Rizal University
Pasig City, Philippines
nanelle.tuguinay@my.jru.edu

***Abstract*—This report classifies the success of k-Nearest Neighbors (kNN) algorithms. The modular kNN model reached an accuracy of 96.67% on the Iris dataset. This research investigates the kNN's usage by the government in the domains of healthcare fraud, geo-technical auditing, and forest fire mapping. kNN has proven to be successful with spatial data, however the use of distance metrics and the black box phenomenon of kNN suggests the need for cumbersome pre-processing and a system of ethics to guarantee a degree of transparency in the use of such algorithms in public sector decision-making.**

Keywords*—k-Nearest Neighbor (kNN), Iris Dataset, Algorithmic Transparency, Government Operations, Classification , Euclidean Distance , Hyperparameter Tuning , Bias-Variance Tradeoff, Confusion Matrix.

I. INTRODUCTION

The use of machine learning in public administration has moved from trial and error to mainstream use. One of the common tools in this transition is the k-Nearest Neighbors (kNN) algorithm, a non-parametric, simple algorithm. kNN is a "lazy learner" that classifies data by looking at the new data and finding the closest training data. In this case, the objective is predicting the species of an Iris based on the measurements of its sepals and petals as represented in a dataset by Ronald Fisher in 1936. The analysis that follows aims to combine the need for prediction (in this case, Iris species prediction) and the need for explanation as a warranted need in this potential application for the public sector.

II. METHODOLOGY

A. Data Preparation and Model Initialization

- Logic: This module takes care of the environment preparation and the retrieval of datasets.
- Implementation: For example, we used the module sklearn.datasets.load_iris which returns 150 examples of three different species.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load data
data = load_iris()

# Separeate features and target
X = data.data
y = data.target

# Splitting the data for training and testing
from sklearn.model_selection import train_test_split
X_Train, X_Test, y_Train, y_Test = train_test_split(X, y, test_size = 0.2, random_state = 0)

# Create a KNeighborsClaasifier object
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors = 5)
```

Fig. 1. (*Data Preparation and Model Initialization Source Code*)

B. Preprocessing (Data Splitting)

- Logic: For model validation, the data was separated into 120 training and 30 testing examples (an 80/20 split).
- Syntax: To achieve random distribution for the features (X) and targets (y), using the train_test_split function.

```
# Train the model using the training data
knn. fit(X_Train, y_Train)

KNeighborsClassifier
KNeighborsClassifier()
```

Fig. 2. (*Preprocessing (Data Splitting) Source Code*)

C. Model Initialization

- Logic: The kNN estimator is initialized by setting the k hyperparameter (number of neighbors).
- Syntax: We created an instance of the class KNeighborsClassifier(n_neighbors=5).

```
# Use the model to predict the test set
predictions = knn.predict(X_Test)
```

Fig. 3. (*Model Initialization Source Code*)

D. Model Training (Fitting)

- Logic: The model learns to associate input features with target labels using "Instance-Based Learning" by retaining the spatial coordinates of the training samples.
- Syntax: The command knn.fit(X_train, y_train) was called to arrange the features in a multi-dimensional space.

```python
# Check accuracy
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_Test,predictions)
print("Accuracy :", accuracy)

# Evaluate model performance
from sklearn.metrics import classification_report
print("classification report")
print(classification_report(y_Test,predictions))

# Making a confusion matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_Test,predictions)

print("Confusion matrix :")
print(cm)
```

```
Accuracy : 0.9666666666666667
classification report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        11
           1       1.00      0.92      0.96        13
           2       0.86      1.00      0.92         6

    accuracy                           0.97        30
   macro avg       0.95      0.97      0.96        30
weighted avg       0.97      0.97      0.97        30

Confusion matrix :
[[11  0  0]
 [ 0 12  1]
 [ 0  0  6]]
```

Fig. 4. (*Model Training (Fitting) Source Code*)

E. Model Inference (Prediction)

- Logic: The model determines the *k* nearest neighbors using the Euclidean distance and labels by majority voting.
- Syntax: The model produced predictions with the command predictions = knn.predict(X_test).

```python
# Visualization of the Scatter plot with predicted classes
plt.figure(figsize = (8, 6))
for i in range(3):
  plt.scatter(X_Test[:, 0][y_Test == i], X_Test[:, 1][y_Test == i], cmap = 'viridis')
plt.scatter(X_Test[:, 0], X_Test[:, 1], c=predictions, cmap='viridis', alpha = 0.8)
plt.xlabel(data.feature_names[0])
plt.ylabel(data.feature_names[1])
plt.title("Scatter Plot with Predicted Classes")
plt.legend()
plt.show()

# Confusion matrix visualization
plt.figure(figsize = (8, 6))
plt.imshow(cm, cmap = 'Blues')
plt.colorbar()
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.xticks(ticks = range(3), labels = data.target_names)
plt.yticks(ticks = range(3), labels = data.target_names)
plt.show()
```

Fig. 5. (*Model Inference (Prediction) Source Code*)

F. Performance Evaluation

- Logic: This module describes the model's accuracy by using the score, classification_report, and confusion_matrix.
- Interpretation: The confusion matrix is used to identify and describe some types of misclassifications (i.e., misclassifying Versicolor for Virginica).

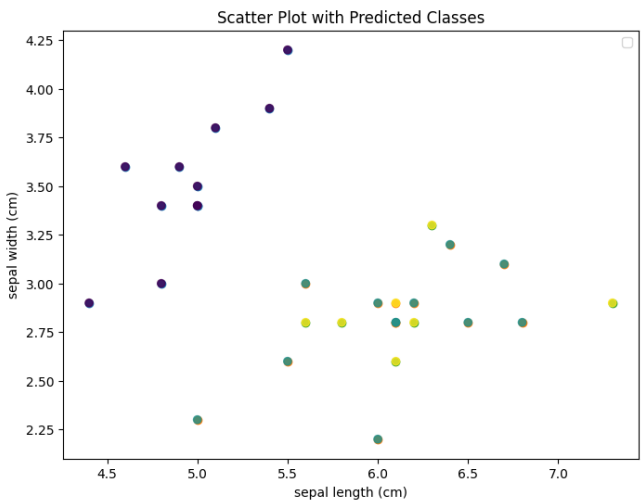

Fig. 6. (*Scatter Plot with Predicted Classes*)

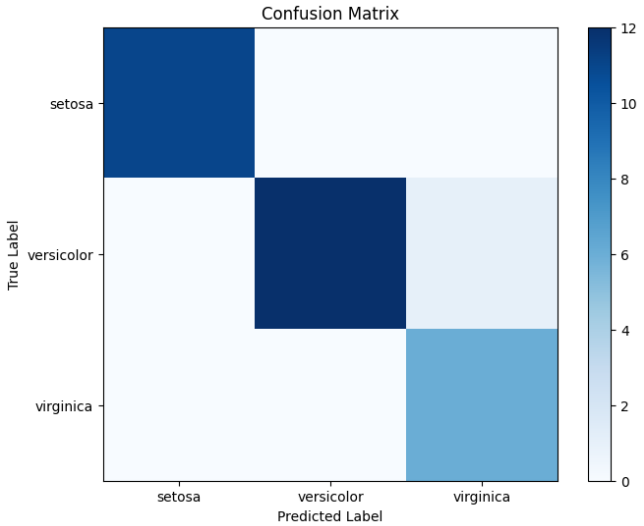

Fig. 7. (*Confusion Matrix*)

III. Results and Experimental Analysis

A. Accuracy and k-Value Experimentation

The experiment analyzed the effect of varying the *k* parameter on model accuracy.

k Value	Performance Summary		
	Accuracy	***Observation***	***Model State***
1	1.00	Zero misclassifications in the test set.	Overfitting
3, 5, 10, 15	0.967	Optimal performance with only 1 error (Versicolor as Virginica).	Optimal
25, 51	0.900	Accuracy begins to degrade as boundaries smooth.	Underfitting
75	0.700	Significant loss in ability to identify Versicolor.	Underfitting
99, 100, 101	0.567	High bias; model defaults to majority class voting.	Severe Underfitting

Fig. 8. (*Performance Summary Table of the experimentation results*)

B. Confusion Matrix Analysis

At *k=5*, the model achieved an accuracy of 96.67%. The confusion matrix showed that Setosa was classified correctly and without any mistakes. Still, one Versicolor was misclassified as Virginica. This shows that kNN has problems when the feature borders are unclear or overlap.

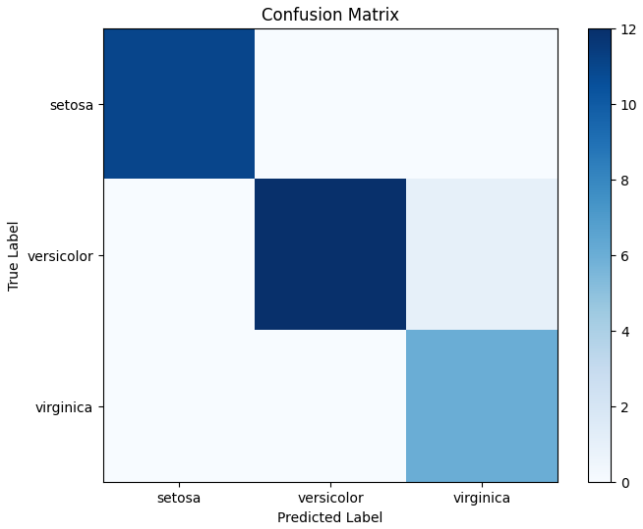

Fig. 9. (*Confusion Matrix of k = 3 (Balanced)*)

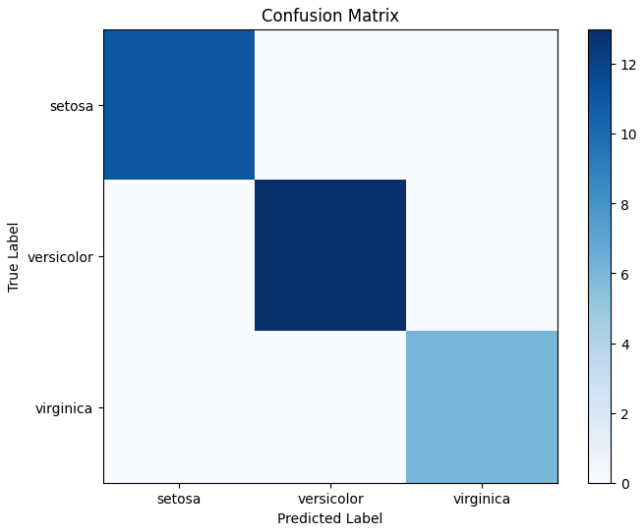

Fig. 10. (*Confusion Matrix of k = 1 (Overfitting)*)

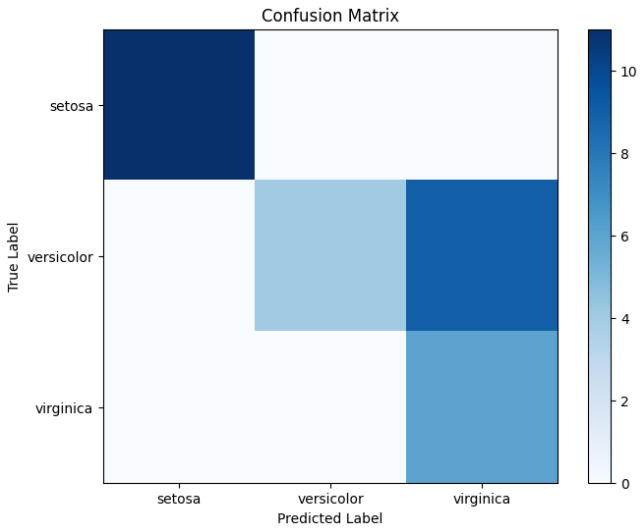

Fig. 11. (*Confusion Matrix of k = 75 (Underfitting)*)

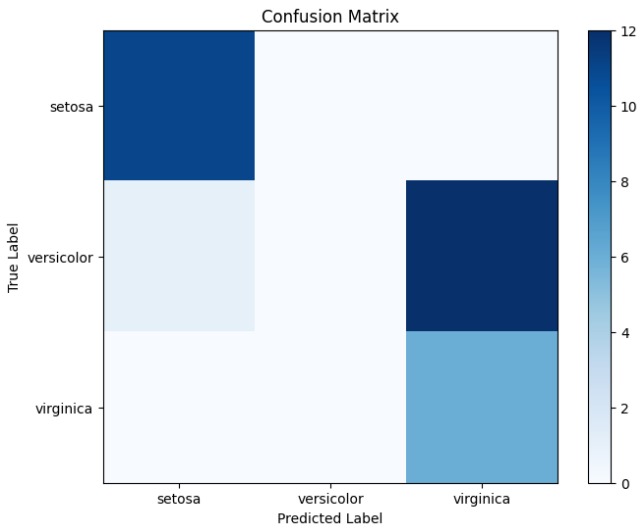

Fig. 12. (*Confusion Matrix of k = 100 (Severe Underfitting - failure)*)

IV. Real-World Applications in Government

A. Operational Investigation

1) Healthcare Fraud: Distinguishing between fraudulent and legitimate billing patterns. [1]

2) Urban Geotechnical Audit: Soil density estimation for construction permits. [2]

3) Forest Fire Mapping: Using satellite imagery to classify land as burned or unburned. [3]

B. Technical Specification Audit

1) Case 1 (Fraud): Classifying billing providers as “Fraudulent” and “Legitimate”. While some models have demonstrated success in this area, research by Obodoekwe and Haar notes that Naive Bayes encounters difficulties with Medicare datasets, which further confirms that kNN should be scrutinized in terms of its particular advantages and disadvantages. [1]

2) Case 2 (Geotechnical): Galupino and Dungca applied Euclidean Distance to assess the proximity of borehole locations. The parameters (Depth, Latitude, Longitude) constituted the basis for this metric for spatial data6. They varied k to examine how it affected the prediction of SPT N-value, similar to our lab experiments. [2]

3) Case 3 (Forest Fire): Pacheco et al. also applied Euclidean Distance since the attributes were spectral bands. They defended their chosen *k* value with ROC Curves and obtained an accuracy of 89% - 93%. [3]

C. Ethical Decision-Making & Transparency

As a "functional black box," kNN has potential risks for use in high-stakes government applications. Using unvalidated kNN in crucial situations such as disaster relief is unwise because unvalidated kNN cannot be used in Iris model situations which have a 3.33% error. What if the model misclassifies a burned farm as "unburned"? aid will be denied with no transparency in the underlying algorithm. kNN should then serve exclusively as a Decision Support System with human oversight.

V. CONCLUSION

kNN proves to be efficient and useful for classifying in the government, and more so for spatial tasks. However, noise in data and overlapping classes require a lot of preprocessing and human reasoning to comply with public service ethics.

REFERENCES

[1] N. Obodoekwe and D. T. van der Haar, "A Comparison of Machine Learning Methods Applicable to Healthcare Claims Fraud Detection," in Advances in Intelligent Systems and Computing, vol. 918, Springer, 2019.

[2] J. Galupino and J. Dungca, "Development of a k-Nearest Neighbor (kNN) Machine Learning Model to Estimate the SPT N-Values of Valenzuela City, Philippines," IOP Conf. Ser.: Earth Environ. Sci., vol. 1091, no. 1, p. 012021, 2022.

[3] A. P. Pacheco et al., "Assessment of k-Nearest Neighbor and Random Forest Classifiers for Mapping Forest Fire Areas in Central Portugal," Remote Sens., vol. 13, no. 7, p. 1345, 2021.